Indigo ESB 서버 이중화 가이드

Indigo ESB Platform

METABUILD CO., LTD.

MESIM ESB/CEP/EAI/MCI/mHUB/DDS

R&D Center / ESB division
저작권 ©2019
Metabuild, R&D Center

Indigo ESB 서버 이중화 가이드

Indigo ESB Platform

METABUILD CO., LTD.

MESIM ESB/CEP/EAI/MCI/mHUB/DDS

R&D Center / ESB division
저작권 ©2019
Metabuild, R&D Center

목차

Chapter 01 ESB HA 구성 패턴 – (1) | 6

1.1 개요 | 4

1.2 사용되는 기본개념 | 4

1.2.1 Active 컨슈머 | 4

1.2.2 Subscription(구독) | 4

1.2.3 Subscription의 확산 | 4

1.2.4 Network Connector | 5

1.3 커넥터 설정 유형 별 작동 방식 | 5

1.3.1 일반 | 5

1.3.2 Single connector 의 경우 | 5

1.3.3 양방향 Connetor 의 경우 | 6

1.3.4 Duplex connector 의 경우 | 6

1.3.5 Multiple Connector 의 경우 | 6

1.3.6 conduitSubscriptions 란? | 7

1.4 브로커 측 설정 (유형별) | 9

1.5 Message consumer(수신어댑터) 측 설정 | 13

Chapter 02 ESB HA 구성 패턴 – (2) | 16

2.1 개요 | 16

2.2 작동 방식 | 16

2.2.1 초기상태 | 16

2.2.2 Master 의 중단 | 17

2.2.3 셧다운 되었던 처음의 master 가 다시 기동되었을 때 | 17

2.3 브로커 측 설정 (1) – 공유 DB 이용 방식 | 18

2.4 브로커 측 설정 (2) – 공유 디스크 이용 방식 | 20

2.5 Message consumer(수신어댑터) 측 설정 | 21

2.6 공유 DB 사용 방식에서의 PERSISTENCE DB 테이블 확인 | 21

Chapter 01

ESB HA 구성 패턴 – (1)

1.1 개요

1.2 사용되는 기본개념

1.3 커넥터 설정 유형 별 작동 방식

1.4 브로커 측 설정 (유형별)

1.5 Message consumer(수신어댑터) 측 설정

1. ESB HA 구성 패턴 – (1)

○ Active-Active 클러스터링

1.1 개요

1. 2 식 이상의 브로커가 브로커 네트워크 [이하 A-A 클러스터라 함]를 형성
2. 각 브로커의 "Network Connector" 의 설정을 통해 구성
3. 개발자는 A-A 클러스터를 하나의 브로커처럼 사용할 수 있게 됨
4. 로드밸런싱(대용량 메시지 처리) 및 무중단 운영(하나의 브로커가 다운되어도 나머지 브로커를 통해 연계가 계속됨) 등의 장점
5. 분배된 메시지를 받은 브로커가 다운되는 경우, 그 메시지는 해당 브로커의 재기동 전까지는 연계가 중단됨. 따라서 메시지 순차보장이 필수적인 상황에서는 A-A 클러스터 구성이 아닌 master/slave 구성을 선택해야 함

1.2 사용되는 기본개념

1.2.1 Active 컨슈머

A-A 클러스터링을 구성하는 브로커들 중 하나에 연결되어, 메시지를 수신할 준비가 되어있는 메시지 컨슈머(수신 어댑터). 자신이 바라보는 큐 또는 토픽에 대한 정보를 연결된 브로커(local 브로커)에 전달.

A-A 클러스터는 Active 컨슈머(수신 어댑터)의 생성/소멸에 대해 계속 추적함

1.2.2 Subscription(구독)

특정 Active 컨슈머(수신 어댑터)가 특정 큐/토픽으로부터 메시지를 수신할 준비가 되어있다는 정보. 브로커는 Subscription 정보를 이용해 어떤 메시지를 어디로 라우팅할지를 결정함. Message selector 에 대한 정보도 여기에 포함

1.2.3 Subscription 의 확산

메시지 컨슈머(수신 어댑터)가 브로커에 접속하면, 그 브로커(local 브로커)에 자신의

Subscription

정보를 넘기게 되며, 그 브로커는 다른 브로커에 그 subscription 정보를 확산함.

1.2.4 Network Connector

A-A 클러스터는 Network connector 를 이용하여, 브로커 간에 커넥션을 정의함으로써 구성. 커넥션의 종류로는 단방향(single), 양방향(duplex), 양방향(single+single)이 있음.

1.3 커넥터 설정 유형 별 작동 방식

1.3.1 일반

브로커"A"에서 브로커 "B"로 Network Connector 가 설정되면, "A"는 "B"의 데스티네이션(큐)들 및 그 데스티네이션을 바라보고 있는 Message Consumer(수신 어댑터)에 대한 정보를 얻음

각 브로커 이벤트에 대해 생성되는 Advisory 메시지에 이러한 정보가 담겨있고, 이 정보를 이용하여, "A"는 "B"로 메시지를 라우팅

1.3.2 Single connector 의 경우

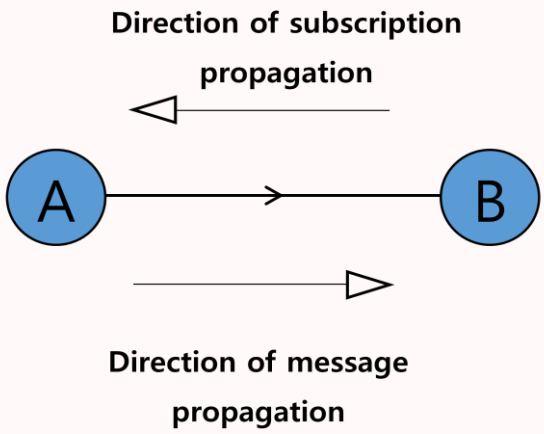

그림 1

- 위 그림은 브로커 "A" 쪽의 설정파일에 설정한 경우.

- 브로커 "A"에서 브로커 "B"로 커넥터를 설정한 경우(single) B 에 연결된 Message Consumer 에 대한 정보(subscription)은 A 로 확산되어, "B 의 A 에 대한 subscription 정보"가 추가됨.
- 반대로 A 브로커로 들어온 메시지는 B 브로커로 전달됨(단방향).
- 따라서 Message producer(송신 어댑터)는 A 에, Messageconsumer(수신 어댑터)는 B 에 연결되는 것이 권장됨. [B 에 들어온 메시지는 A 에 연결된 어댑터가 수신 못함]

1.3.3 양방향 connector 의 경우

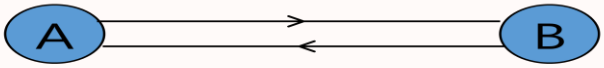

그림 2

- Single connector 를 중첩하여 설정(브로커 A 및 B 설정파일 모두에 각각 설정)
- 메시지는 양방향으로 전달될 수 있음
- 송신, 수신 어댑터는 브로커 A, B 중 어느 쪽이나 임의연결하더라도 메시징 문제 없음.

1.3.4 Duplex connector 의 경우

- 브로커 A 또는 B 한 쪽에만 설정하면 됨.
- 메시지는 양방향으로 전달될 수 있음
- A, B 간의 통신을 위해 방화벽 포트 오픈 작업이 필요한 경우, 오직 1 개의 포트로 쌍방향 통신이 가능하다는 이점. (다만, 보통의 A-A 클러스터 구성시 A,B 모두 같은 내부단에 위치하게 될 것이므로 논의의 실익은 적음)

1.3.5 Multiple connector 의 경우

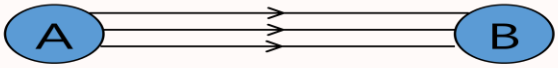

그림 3

- 각 커넥터에 고유한 이름을 주어 단방향 커넥터를 여러 개 설정 가능
- 메시징 로드를 분산시킬 수 있음
- 한 개의 커넥터는 큐 통신에, 다른 한 개의 커넥터는 토픽 통신에 이용하도록 설정할 수 있음.

1.3.6 conduitSubscriptions 란?

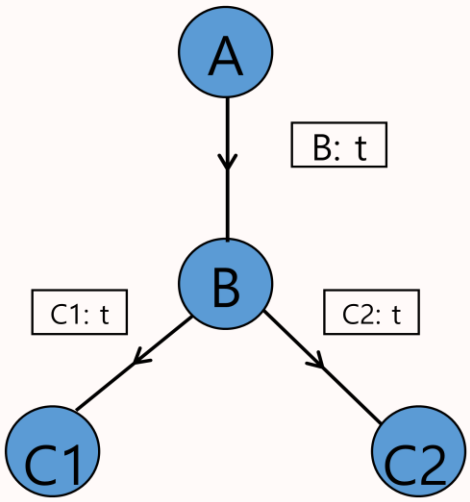

그림 4

- conduitSubscription 이란 동일한 큐 또는 토픽으로의 subscription 들을 하나의 subscription 으로 통합하여 인식하는 것
- 본 옵션을 enable 할 지의 여부는 해당 subscription 이 큐 대상인지 토픽 대상인지에 따라 달라지므로 잘 판단해야.

○ Case #1 : 토픽 & conduitSubscriptions="true"

위의 그림에서 message consumer c1, c2 는 동일 브로커 ["B"]의 동일한 데스티네이션[토픽 "t"]로의 subscription 을 각각 생성[c1:t,c2:t]함. 그런데, 브로커 "A"에 conduitSubscriptions="true"로 설정된 경우, 앞서 언급한 두개의 subscription 을 통합하여 하나의 subscription B:t 를 구성한 후, 이를 "A"에 등록. 이후, "A"(의 토픽 "t")에 메시지가 들어오면 그메시지는 일단, B 의 토픽 "t"로 이동하고, 다시 토픽의 전송원리에 따라 c1, c2 로 메시지 전송

○ Case #2 : 토픽 & conduitSubscriptions = "false"

브로커 A 에는 두 개의 subsctiption 이 등록 (B:c1:t, B:c2:t). 이후, A 의 토픽"t" 에 메시지가 들어오면, 일단, B 로 2 건의 메시지 이동(subscription 이 2 개이므로), 이후 토픽의 전송 원리에 따라 c1, c2 로 메시지 전송되므로, c1, c2 는 각각 2 건의 메시지를 전송받게 되어 문제

○ Case #3 : 큐 & conduitSubscriptions = "true"

message consumer c1, c2 는 동일 브로커 ["B"]의 동일한 데스티네이션[큐 "t"]로의 subscription 을 각각 생성[c1:t, c2:t]함. (이 경우, message selector 를 사용). 그런데, 브로커 "A"에 conduitSubscriptions="true"로 설정된 경우, 앞서 언급한 두개의 subscription 을 통합하여 하나의 subscription B:t 를 구성하면서, 이를 "A"에 등록.(이 때, messageSelector 에 대한 정보 소멸됨) 이후, "A"(의 큐 "t")에 메시지가 들어오면 그메시지는 일단, B 의 큐 "t"로 이동하고, 다시 큐의 전송원리에 따라 c1 또는 c2 로 메시지 전송(messageSelector 의 적용있음)

주의 : 통합/등록되는 것은 A 에 등록되는 subscription 일 뿐이고, B 에 등록된 최초의 message consumer 의 subscription 2 개는 그대로 유지되므로 B 로 들어온 메시지에는 message selector 가 작용

○ Case #4 : 큐 & conduitSubscriptions = "false"

message consumer c1, c2 는 동일 브로커 ["B"]의 동일한 데스티네이션[큐 "t"]로의 subscription 을 각각 생성[c1:t, c2:t]함. (이 들은, message selector 있음). 브로커 "A"가 2개 conduitSubscriptions="false"로 설정된 경우, subscription B:C1:t, B:C2:t 가 각각 구성되면서, 이들 모두 "A"에 등록.(이 때, 각 subscription 의 messageSelector 에 대한 정보가 보존됨) 이후, "A"(의 큐 "t")에 메시지가 들어오면 그 메시지(1 건)는 subscription "B:c1:t"에 의해(이 subscription 에는 c1 이 수신할 수 있도록 message selector 가 설정되어 있음) B 의 큐 "t"로 이동하고, 다시 큐의 전송원리에 따라 c1 으로 메시지 전송[송신 어댑터에서는 message selector 를 이용할 수 있도록 JMS property(c1)를 세팅하여 송신하였다고 가정]

○ 종합 : case 3, case 4 최종결과는 같음

다만, Case3 의 경우, 브로커 A 에서 브로커 B 로 메시지 이동 시에도 message selector 의 적용을 받으나 (message selector 총 2 번 적용), Case 4 의 경우는, 일단 selector 적용 없이 B 로 옮기고, 이후 message consumer 가 수신할 때에만 message selector 를 작용할지(총 1 번 적용)의 차이만 있음

1.4 브로커 측 설정 (유형별)

각 브로커의 설정파일 “indigomq.xml” 에 설정

엘리먼트 “NetworkConnectors” 아래의 “NetworkConnector” 엘리먼트에 구성

○ Single connector 의 경우

```
<beans ...>
    <broker xmlns="http://activemq.apache.org/schema/core"
            brokerName="brokerA" brokerId="A" ... >
        ...
        <networkConnectors>
            <networkConnector name="linkToBrokerB" uri="static:(tcp://localhost:61002)"
                              conduitSubscriptions="false"/>
        </networkConnectors>
        ...
        <transportConnectors>
            <transportConnector name="openwire" uri="tcp://0.0.0.0:61001"/>
        </transportConnectors>
    </broker>
</beans>
```

브로커 A(0.0.0.0:61001)에 위와 같이 설정하는 경우, A 에서 B(localhost:61002)로의 single connector 가 설정됨

○ 양방향 connector 의 경우

```
<beans ...>
    <broker xmlns="http://activemq.apache.org/schema/core"
            brokerName="brokerA" brokerId="A" ... >
        ...
        <networkConnectors>
            <networkConnector name="linkToBrokerB" uri="static:(tcp://localhost:61002)"
                conduitSubscriptions="false" />
        </networkConnectors>
        ...
        <transportConnectors>
            <transportConnector name="openwire" uri="tcp://0.0.0.0:61001"/>
        </transportConnectors>
    </broker>
</beans>
```

```
<beans ...>
    <broker xmlns="http://activemq.apache.org/schema/core"
            brokerName="brokerB" brokerId="B" ... >
        ...
        <networkConnectors>
            <networkConnector name="linkToBrokerA" uri="static:(tcp://localhost:61001)"
                conduitSubscriptions="false"/>
        </networkConnectors>
        ...
        <transportConnectors>
```

```
            <transportConnector name="openwire" uri="tcp://0.0.0.0:61002"/>

        </transportConnectors>

    </broker>

</beans>
```

양방향 connector 설정은 위와 같이 관계된 모든 브로커에 동시에 설정되어야 함

○ Duplex 설정의 경우

```
<beans ...>

    <broker xmlns="http://activemq.apache.org/schema/core"

            brokerName="brokerA" brokerId="A" ... >

        ...

        <networkConnectors>

            <networkConnector name="linkToBrokerB" uri="static:(tcp://localhost:61002)"

                    duplex="true" conduitSubscriptions="false"/>

        </networkConnectors>

        ...

        <transportConnectors>

            <transportConnector name="openwire" uri="tcp://0.0.0.0:61001"/>

        </transportConnectors>

    </broker>

</beans>
```

```
<beans ...>
```

```
    <broker xmlns="http://activemq.apache.org/schema/core"
                brokerName="brokerB" brokerId="B" ... >
...
            <transportConnectors>
                <transportConnector name="openwire" uri="tcp://0.0.0.0:61001"/>
            </transportConnectors>
    </broker>
</beans>
```

Network Connector 는 관련된 브로커들 중 한 쪽에만 설정하면 됨.

○ 단방향 복수개(multiple) connector 의 경우

```
<beans ...>
    <broker xmlns="http://activemq.apache.org/schema/core"
                brokerName="brokerA" brokerId="A" ... >
            ...
<networkConnectors>
   <networkConnector name="link01ToBrokerB" uri="static:(tcp://localhost:61002)"
            conduitSubscriptions="false"/>
   <networkConnector name="link02ToBrokerB" uri="static:(tcp://localhost:61002)"
            conduitSubscriptions="false"/>
   <networkConnector name="link03ToBrokerB" uri="static:(tcp://localhost:61002)"
            conduitSubscriptions="false"/>
</networkConnectors>
```

```
<transportConnectors>

    <transportConnector name="openwire" uri="tcp://0.0.0.0:61001"/>

</transportConnectors>

</broker>
```

```
<beans ...>

    <broker xmlns="http://activemq.apache.org/schema/core"

            brokerName="brokerB" brokerId="B" ... >

...

      - <transportConnectors> 고유한 name 을 가져야 함각 connector 는

            <transportConnector name="openwire" uri="tcp://0.0.0.0:61001"/>

        </transportConnectors>

    </broker>

</beans>
```

1.5 Message consumer(수신어댑터) 측 설정

○ 양방향 connector 설정시,

1. 수신어댑터에 1 개의 messageListener 를 설정하고, 1 개의 브로커만 등록한 경우. failover:(tcp://broker1:61616) 브로커 A 에 들어온 메시지를 브로커 B 를 바라보는 수신어댑터에서 처리 가능. 반대로 브로커 B 에 들어온 메시지도 브로커 A 를 바라보는 수신어댑터에서 처리가능. 그런데, 이 경우, 바라보고 있는 브로커가 다운된 경우, 연계 중단됨

2. 수신어댑터에 messageListener 를 2 개 설정하는 경우 1 의 경우와 달리 A-A 클러스터를 구성하고 있는 브로커들 중 하나가 다운되도 연계는 지속됨 그러나, 수신어댑터가 계속하여 다운된 브로커로 연결시도하여 익셉션을 발생시킬 것임

3. 수신어댑터에 1 개의 messageListener 를 설정하고, failover 전송방식을 이용 2 개의 브로커를 등록한 경우 failover:(tcp://broker1:61616,tcp://broker2:61616) 기본적인 기능은 1 의 경우와 같으며, 브로커 중 하나가 다운된 경우, 다른 브로커로 연결하므로 연계 지속

4. 결론 -> 방법 3 사용

Chapter 02

ESB HA 구성 패턴 – (2)

2.1 개요

2.2 작동 방식

2.3 브로커 측 설정 (1) – 공유 DB 이용 방식

2.4 브로커 측 설정 (2) – 공유 디스크 이용 방식

2.5 Message consumer(수신어댑터) 측 설정

2.6 공유 DB 사용 방식에서의 PERSISTENCE DB 테이블 확인

2. ESB HA 구성 패턴 – (2)

○ Active-StandBy (Master/Slave)

2.1 개요

1. 2 식 이상의 브로커가 master/slave 그룹을 형성
2. JDBC master/slave 설정은 브로커들의 JDBC persistence adapter 부분의 설정을 통해 수행
3. 공유 디스크 master/slave 설정은 브로커들의 imqPersistenceAdaptor 의 설정의 통해 수행
4. 그룹에 속한 브로커 중 어떤 것이 master 가 될지는 DB 테이블(공유 디스크)에 대한 락(lock)의 소유여부에 있음

2.2 작동 방식

2.2.1 초기상태

2 식의 브로커가 모두 실행되지만, 그 중 하나가 DB 테이블(또는 공유 디스크)에 대한 배타적인 lock 을 잡아 master 가 됨.

이렇게 Master 가 된 브로커만이 transport connector 를 시작하여 클라이언트를 붙일 수 있게 됨.

Slave 는 lock 을 소유하여 작동이 시작될 때까지 기다리면서 정지상태 유지

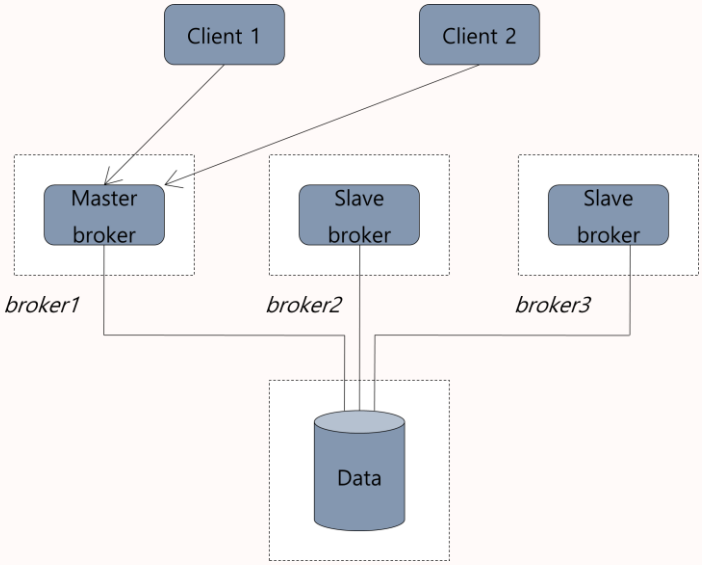

그림 5

2.2.2 Master 의 중단

Master 브로커가 셧 다운되면, 테이블(또는 공유 디스크) lock 을 놓게 되며(즉시 또는 설정된 timeout 후) 이를 slave 인 브로커가 잡게 되면 새로운 master 로 변모. 처음의 Master 와 연결되어 있던 클라이언트가 커넥션을 잃게 되면, failoverURL 에 설정되어 있는 다른 모든 브로커에 대해 자동으로 연결을 시도하다가 새롭게 master 가 된 브로커에 최종 연결됨

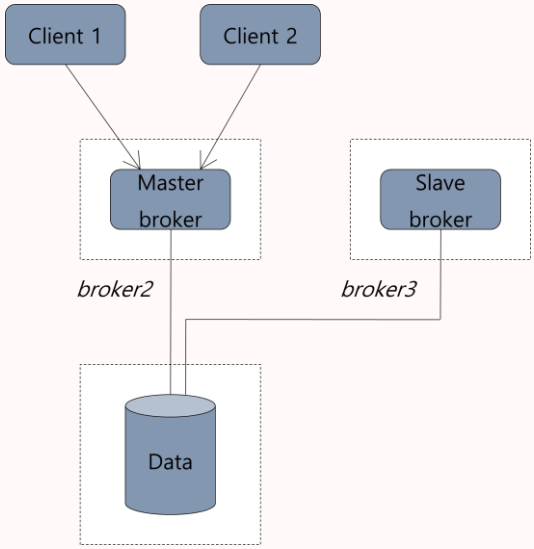

그림 6

2.2.3 셧다운 되었던 처음의 master 가 다시 기동되었을 때

다시 기동된 브로커는 slave 로서 그룹에 포함. 차후 다시 DB 테이블(또는 공유 디스크) lock 을

소유하게 되면, master 화하게 됨.

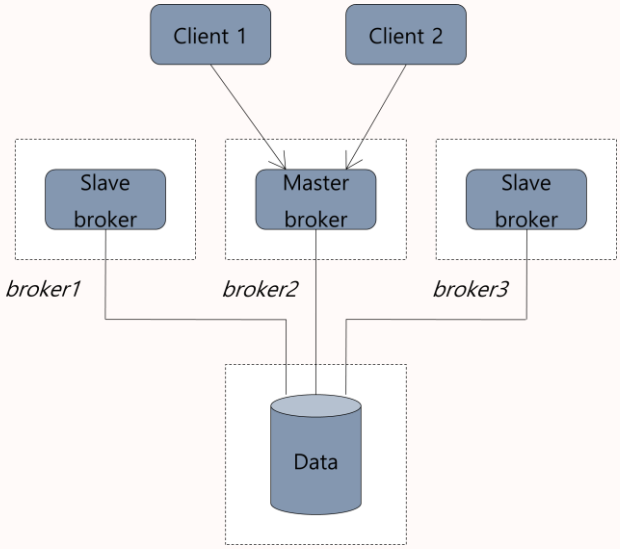

그림 7

2.3 브로커 측 설정 (1) – 공유 DB 이용 방식

- JDBC master/slave 설정에 있어, master 와 slave 측 설정은 동일
- 각 브로커의 {$ESB_HOME}/conf/indigomq.xml 에서 설정
- Master/slave group 에 속하는 모든 브로커는 동일한 JDBC persistence Adaptor 를 사용하고, 동일한 DB 를 이용하도록 설정되어야 함.

참고 : 단일 (또는 active-active(확인 필요)) 브로커 구성에서 JDBC persistence adapter 를 이용하는 경우, 고속의 저널(Journal)을 이용할 수 있으나, master/slave 설정에서는 반드시 direct 방식의 JDBC persistence adapter 을 이용해야 함. 저장되는 데이터가 최신본이 아닐 수 있음). (이유: journaled JDBC persistence Adaptor 는 JDBC DB 에 JDBC DB 와의 동기화를 위해 저널을 복사하지 않고, batch update 를 이용하므로,

○ 다음은 master 측 설정의 예시.

slave 측도 동일하게 설정

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:amq="http://activemq.apache.org/schema/core"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
                    http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
                    http://activemq.apache.org/schema/core
                    http://activemq.apache.org/schema/core/activemq-core-5.3.1.xsd">
    <broker xmlns=http://activemq.apache.org/schema/core
     brokerName="brokerA">
  ..
        <persistenceAdapter>
            <jdbcPersistenceAdapter dataSource="#oracle-ds"/>
        </persistenceAdapter>
  ...
    </broker>
 <bean id="oracle-ds"
  class="org.apache.commons.dbcp.BasicDataSource"
  destroy-method="close">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:AMQDB"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
        <property name="poolPreparedStatements" value="true"/>
</bean>
</beans>
```

2.4 브로커 측 설정 (2) – 공유 디스크 이용 방식

- 공유디스크 이용 방식 master/slave 설정에 있어, master 와 slave 측 설정은 동일
- 각 브로커의 {$ESB_HOME}/conf/indigomq.xml 에서 설정
- Master/slave group 에 속하는 모든 브로커는 동일한 imqPersistenceAdaptor 를 사용하고, 동일한 디스크를 이용하도록 설정되어야 함.

○ 다음은 master 측 설정의 예시. slave 측도 동일하게 설정

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:amq="http://activemq.apache.org/schema/core"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
                        http://activemq.apache.org/schema/core
                        http://activemq.apache.org/schema/core/activemq-core-5.3.1.xsd">
    <broker xmlns="http://activemq.apache.org/schema/core"
                    brokerName="brokerA">
  ...
        <imq:persistenceAdapter>
            <imq:imqPersistenceAdapter syncOnWrite="false" maxFileLength="500mb"
             directory="/was/ESB/data1"/>
        </imq:persistenceAdapter>
...
    </broker>
</beans>
```

2.5 Message consumer(수신어댑터) 측 설정

master/slave 그룹에 연결되는 클라이언트는 그룹에 속한 모든 브로커를 망라하는 failover URL 을 사용하여 접속 구성.

○ 다음은 설정 예시. 클라이언트 MQconnectionFactory 설정의 "brokerUrl"에 기재

failover:(tcp://broker1:61616,tcp://broker2:61616,tcp://broker3:61616)

2.6 공유 DB 사용 방식에서의 PERSISTENCE DB 테이블 확인

설정된 DB 에는 테이블이 3 개 생성

Persistent 방식 전송 이용시 INDIGOMQ_MSGS 테이블이 의미 있음

테이블	기능
INDIGOMQ_MSGS	PERSIST 된 데이터들이 컨수머측으로부터 ACK 받기 전까지 저장됨
INDIGOMQ_LOCK	새로운 MASTER 가 생기는 이벤트(LOCK 을 가지게되는)를 기록하는 것으로 추정 (현재 어떤 이유에서인지 데이터가 입력되지 않아 원인 파악 중)
INDIGOMQ_ACKS	TOPIC 이용 연계시, durable Subscription 을 위한 데이터 저장